Nombre: Fecha:

Ángel Nahúm CS 15/octubre/2026

Conceptos de Estructuras Dinámicas en Java

1. Definición de Estructuras

Una estructura de datos dinámica es aquella cuyo tamaño puede crecer o decrecer durante la ejecución del programa. A diferencia de las estructuras estáticas, la memoria se solicita y se libera conforme se necesita.

En Java, cuando queremos crear un nuevo objeto o nodo en tiempo de ejecución, utilizamos la palabra reservada new. Este espacio de memoria se asigna en una región llamada heap. Las conexiones entre los elementos no dependen de su posición física, sino de las referencias que guardan la dirección del siguiente elemento. Cuando un nodo ya no es accesible, el Garbage Corrector se encarga de liberar esa memoria automáticamente.

> **Nota Importante**
> El acceso en una lista dinámica es secuencial. Para llegar al elemento n, debemos recorrer todos los anteriores, lo que resulta en una complejidad de O(n) .

2. Características Principales

Completa la siguiente lista de características clave de las estructuras dinámicas basándote en la lectura:

Tamaño variable: Se ajusta a la cantidad real de datos en uso.

- Memoria no contigua: Los elementos pueden estar dispersos y se conectan mediante enlaces.
- Eficiencia: Solo se reserva el espacio estrictamente necesario.
- Flexibilidad: La inserción y eliminación no requieren desplazar otros elementos.
- Costo de memoria: Cada nodo requiere espacio extra para almacenar sus enlaces.

3. Comparación: Arreglo vs. Lista Enlazada

Diffit diffit.me/s/zjxj68t

Analiza las diferencias técnicas entre ambas estructuras y completa los espacios vacíos en la tabla.

Aspecto	Arreglo (Estático)	Lista Enlazada (Dinámica)
Tamaño	Fijo (definido al inicio)	Variable que crece o decrece en ejecución
Acceso por índice	0(1) (Directo)	0(1) (Secuencial, hay que recorrer los nodos anteriores)
Insertar al inicio	O(n) (Requiere desplazar)	0(1) (Solo se reasignan referencias)
Uso de Memoria	Bloque contiguo	Nodos dispersos en el heap, unidos por referencias

4. Análisis de Implementación

¿Por qué es crítico el orden al reasignar referencias durante una inserción? Explica qué sucedería si desconectas el nodo anterior antes de conectar el nuevo nodo a la lista.

5. Si una estructura requiere recorrer todos los elementos anteriores para encontrar el dato buscado, ¿cuál es su complejidad temporal?

A) O(log n)

B) 0(1)

C) O(n)

Diagramación de Referencias y Nodos

> **Regla de Oro**
> Para no perder el acceso al resto de la lista, primero conecta el nuevo nodo a la secuencia existente y después actualiza la referencia previa. Si desconectas antes de asegurar el enlace, el Garbage Collector podría eliminar los nodos huérfanos.

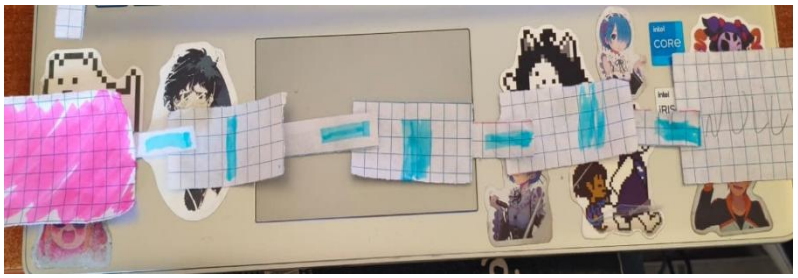
En una lista enlazada simple, cada nodo tiene un valor y una referencia al siguiente nodo (next. A continuación, realizarás el

Nombre: _____ Fecha: _____

trazado visual de cómo cambian estas referencias durante operaciones comunes.

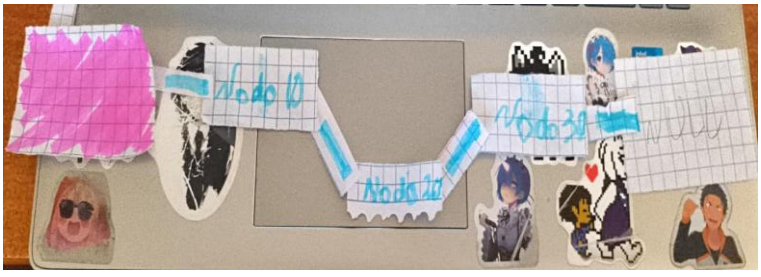
Escenario 1: Insertar al inicio

Se tiene una lista con los valores [B] -> [C] . Se desea insertar un nuevo nodo [A] al principio de la lista. Dibuja cómo deben quedar las flechas de referencia.

Estado Inicial (Antes)	Tu Diagrama (Después)
cabeza Nodo B Nodo C null	 Cabeza -> Nodo A -> Nodo B -> Nodo C -> NULL

Escenario 2: Insertar entre dos nodos

Se tiene la lista [1@] -> [3@]. Se desea insertar el nodo [2@] entre el 10 y el 30.
Muestra el orden correcto de las conexiones para no perder el nodo 30.

Estado Inicial (Antes)	Tu Diagrama (Después)
Nodo 10 Nodo 30 null	 Cabeza -> Nodo 10 -> Nodo 20 -> Nodo 30 -> NULL

Escenario 3: Eliminar el último nodo

Se tiene la lista [X] -> [Y] -> [Z]. Se desea eliminar el nodo [Z]. Dibuja qué referencia debe cambiar a null y qué sucede con el puntero del nodo penúltimo.

Estado Inicial (Antes)	Tu Diagrama (Después)
Nodo X Nodo Y Nodo Z null	 [X] -> [Y] -> NULL

Pregunta de Reflexión:
¿Por qué en el Escenario 2 es vital conectar [2@]. next= [30] antes de hacer [10]. next=

Es vital porque si primero actualizamos la referencia del nodo anterior ([10].next = [20]) antes de enlazar el nuevo nodo con el resto de la lista, perderíamos la única referencia que tenemos hacia el nodo 30. Al romperse ese enlace directo con el nodo 30, este último quedaría desconectado e inaccesible, provocando que el resto de la estructura se pierda y sea eliminada por el *Garbage Collector*. Primero debemos asegurar el enlace hacia adelante ([20] -> [30]) y después conectar el nodo previo ([10] -> [20]) para mantener la integridad de la lista completa.

Laboratorio de Programación: Implementación de una Lista Enlazada

En esta actividad, aplicarás tus conocimientos sobre estructuras dinámicas para construir una lista enlazada simple en Java. Deberás completar el código siguiendo los comentarios TODO. Recuerda que, en una estructura dinámica, la memoria se gestiona mediante referencias en el heap.

Nombre: _____ Fecha: _____

Recordatorio de Lógica

1. Nodo: Es el bloque básico que contiene el dato y el enlace al siguiente.
2. Referencia: En Java, usamos variables de objeto para apuntar a la dirección de memoria del siguiente nodo.
3. Orden de Inserción: Para no perder la lista, primero conecta tu nuevo nodo a la cabeza actual y luego actualiza la cabeza.

Parte 1: La Clase Nodo

Completa el constructor para inicializar el nodo con un valor entero.

```
public    class
Nodo {int dato;
Nodo siguiente;

//    TODO:    Implementar    el
constructor public Nodo (int
dato) {
```

Parte 2: La Clase ListaSimple

Implementa los métodos para gestionar la estructura. Presta especial atención al método insertarAlInicio para asegurar que la complejidad sea 0(1).

```
public           class
ListaSimple { private
Nodo cabeza;

public ListaSimple()
{ this.cabeza =
null;


//       TODO:       Implementar
insertarAlInicio  public  void
insertarAlInicio(int valor) {


//  TODO:  Implementar  recorrer  (imprimir
todos los elementos) public void recorrer()
{ Nodo actual = cabeza;
```

Parte 3: Análisis de Casos Límite

Una vez completado el código, analiza cómo se comportaría tu implementación en las siguientes situaciones.

Caso Límite	¿Qué sucede en tu código?
La lista está vacía (cabeza null)	El código evalúa que la cabeza es nula, ya que al recorrer detecta que la cabeza es nula e indica que la lista esta vacía sin generar errores nulos.
La lista tiene un solo elemento	El nuevo nodo insertado al inicio apunta su referencia siguiente hacia el nodo que antes era la cabeza (el único existente), y la cabeza se actualiza al nuevo nodo.
Insertar un nuevo nodo cuando ya existen otros	Se cumple la regla de que el nuevo nodo apunta al nodo al que actualmente apunta la cabeza, y luego la cabeza se desplaza al nuevo nodo, y no se pierde los nodos anteriores.

Pregunta de Reflexión

¿Por qué es necesario usar una variable temporal (como actual) en el método recorrer en lugar de mover directamente la variable cabeza? Porque la cabeza es el único punto de acceso al inicio de nuestra estructura enlazada. Si moviéramos directamente la variable cabeza conforme avanzamos en el bucle para imprimir los datos, al terminar el recorrido perderíamos la referencia al primer nodo de la lista (la cabeza original), dejando la estructura inaccesible.

Evaluación: Estructuras Dinámicas en Java

Parte 1: Análisis de Complejidad (Big O)

Compara el rendimiento de los arreglos estáticos frente a las listas enlazadas dinámicas seleccionando la respuesta correcta.

1. ¿Cuál es la complejidad temporal para acceder a un elemento por su índice en una lista enlazada?

A) O(1)

B) O(n)

C) O(log n)

Nombre: _____ Fecha: _____

2. Al insertar un elemento al inicio, ¿qué estructura es más eficiente?

A) El arreglo, porque tiene acceso 0(1) .
B) La lista enlazada, porque solo requiere reasignar referencias 0(1) .
C) Ambas tienen la misma eficiencia O(n) .

3. ¿Por qué un arreglo tiene una complejidad de 0(1) para el acceso por índice?

A) Porque utiliza memoria no contigua.
B) Porque el Garbage Collector optimiza la búsqueda.
C) Porque al estar en memoria contigua, se puede calcular la dirección exacta del dato.

4. Si necesitas buscar un dato específico recorriendo todos los elementos anteriores, la complejidad es:

A) O(n)
B)
0(1)
C)
O(n2)

5. ¿Cuál es una desventaja de la lista enlazada en términos de eficiencia de memoria?

A) La memoria es estática y no se puede liberar.
B) Ocupa menos espacio que un arreglo.
C) Cada nodo consume memoria extra para almacenar la referencia al siguiente elemento.

Parte 2: Gestión de Memoria en Java

Responde Verdadero (V) o Falso (F) a las siguientes afirmaciones sobre el manejo de memoria.

6. En Java, los objetos creados con la palabra new se almacenan en una región de memoria llamada heap.

A) Verdadero

B) Falso

7. El Garbage Co//ector libera automáticamente la memoria de los nodos que aún tienen una referencia activa apuntando a ellos.

A) Falso
B) Verdadero

8. Una estructura dinámica se diferencia de una estática porque su tamaño puede crecer o decrecer durante la ejecución.

A) Falso
B) Verdadero

Parte 3: Lógica de Referencias y Fugas de Memoria

Responde de manera detallada a las siguientes preguntas sobre la manipulación de nodos.

9. Explica por qué es crítico conectar el nuevo nodo a la lista antes de actualizar la referencia de la 'cabeza' durante una inserción al inicio.

Porque si actualizamos la cabeza antes de enlazar el nuevo nodo con el resto de la secuencia, rompemos el acceso a los elementos subsiguientes, provocando que el resto de la lista quede desconectada y sea eliminada por el recolector de basura (*Garbage Collector*).

10. ¿Qué sucede con la memoria en Java si desconectas accidentalmente el primer nodo de una lista sin haber guardado una referencia al resto de los elementos? Justifica tu respuesta mencionando el concepto de Garbage Conector.

Ocurre una pérdida de memoria lógica o "fuga" (*memory leak* desde la perspectiva de la aplicación), ya que al no existir ninguna variable o referencia activa que apunte a esos nodos, el Garbage Collector los detectará como objetos inalcanzables y procederá a destruir y liberar automáticamente ese espacio de memoria en el heap, perdiendo los datos guardados en ellos.